

A Search Reduction from Dominating Set to Leaf-Rich Spanning Tree

Reviewed reconstruction — awaiting expert review
2026-09-22

Abstract

We give a deterministic search reduction from Dominating Set to Maximum Leaf Spanning Tree with a threshold. The construction replaces each source vertex by a set vertex and an element vertex, joins the two layers by closed-neighborhood incidence, and adds a rooted connector with one pendant vertex. A spanning tree with sufficiently many leaves has few non-leaf vertices. The forced non-leaf root accounts for one of them; the remaining non-leaves recover a dominating set. The decoder works for every valid target tree and for the exact NO-SOLUTION output. On an n -vertex, m -edge source graph, the target has $2n + 2$ vertices and $1 + 2n + 2m$ edges. The forward and recovery maps run in polynomial time and accept integers of unbounded encoded length. Exhaustive finite checks and an independent review support the implementation; the general claim rests on the proof.

1. Introduction

A maximum-leaf spanning tree exposes many vertices as endpoints while retaining global connectivity. Its internal vertices form a connected dominating set, so the problem is closely tied to connected domination [2]. Ordinary Dominating Set does not impose connectivity. This difference prevents the identity map from serving as a reduction from the ordinary problem.

Garey and Johnson list a transformation from Dominating Set to Maximum Leaf Spanning Tree [1, problem ND2]. Li and Toulouse use a related set-element incidence graph in a Set Cover reduction for a bipartite leaf problem [3, Theorems 1.3 and 2.1]. These sources establish the surrounding decision-problem ideas, but the present task requires an executable search rule: it must recover a valid source output from every valid target output, including NO-SOLUTION.

This paper reconstructs that rule explicitly. The forward map uses the closed neighborhoods of the source graph as a set system. A pendant vertex forces the connector root to consume one non-leaf position. The decoder maps every other non-leaf in either incidence layer back to a source vertex. This is our main result: the resulting set dominates the source graph and never exceeds the source threshold, regardless of which qualifying target tree is returned.

Theorem 1 (search reduction). Let G be any finite connected simple graph and let k be any nonnegative integer. There are deterministic polynomial-time maps F and R such that $F(G, k)$ is a legal Maximum Leaf Spanning Tree threshold instance and, for every valid target output y , $R((G, k), y)$ is a valid Dominating Set output. Both maps include exact NO-SOLUTION semantics.

The construction is a direct refinement of known incidence ideas, not a new hardness classification. Its contribution is the complete every-valid-output decoder, exact boundary semantics, implementation, and reproducible verification required by Theorem 1.

2. Problem and reduction semantics

Let $G = (V, E)$ be a finite connected simple graph with $V = \{0, \dots, n-1\}$ and $n \geq 1$. The closed neighborhood of v is $N_{G[v]} = \{v\} \cup \{u \in V : \{u, v\} \in E\}$. A valid Dominating Set output for (G, k) is a set $D \subseteq V$ with $|D| \leq k$ and $\cup_{v \in D} N_{G[v]} = V$. The output is NO-SOLUTION exactly when no such set exists.

A target instance is a connected simple graph H and a nonnegative threshold K . A valid witness is a spanning tree of H with at least K degree-one vertices. The target output is NO-SOLUTION exactly when

no such tree exists. In the one-vertex tree the sole vertex has degree zero, so it contributes no leaf under this definition.

For either problem, the valid-output set is nonempty because it contains either a witness or NO-SOLUTION. A search reduction consists of a forward map F and a recovery map R satisfying

$$\forall x, \quad \forall y \in S_{B(F(x))}, \quad R(x, y) \in S_{A(x)}. \quad (1)$$

Equation 1 requires recovery from any valid target output. It does not permit the decoder to assume a canonical tree. This is the metric many-one view of multivalued search reductions [4, Definition 2]. Graphs and outputs are explicit. JSON decimal integers are exact, and their length differs from binary length only by a constant factor.

3. Construction and recovery

Fix a legal source instance (G, k) and put $k' = \min(k, n)$. The target graph H contains a root r , its pendant neighbor p , one set vertex s_v for each $v \in V$, and one element vertex e_u for each $u \in V$. Its edges are

$$E(H) = \{rp\} \cup \{rs_v : v \in V\} \cup \{s_v e_u : u \in N_{G[v]}\}. \quad (2)$$

The target threshold is

$$K = 2n + 1 - k'. \quad (3)$$

Figure 1 shows every vertex and edge for a three-vertex path. The same two layers and attachment rules apply for arbitrary G .

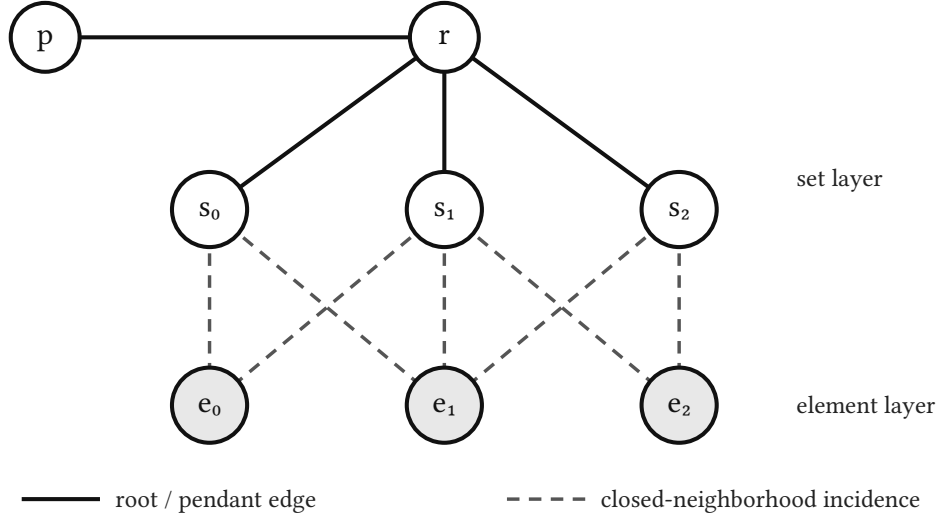


Figure 1: Exact construction for the source path $0 - 1 - 2$. The solid edges attach the pendant p and all set vertices to the root r . A dashed edge $s_v e_u$ is present exactly when $u \in N_{G[v]}$. The shaded element layer is visual only; it does not encode an extra graph property.

The forward algorithm emits the vertices and edges in Equation 2 and the threshold in Equation 3. The cap k' preserves feasibility because V is always a dominating set. It also keeps $K \geq n + 1$, so the target threshold is legal even when the binary encoding of k is arbitrarily long.

The recovery algorithm receives the source instance and a valid target output. It returns source NO-SOLUTION when the target output is NO-SOLUTION. Otherwise, for the returned tree T , it forms

$$D_T = \{v \in V : \deg_{T(s_v)} \geq 2\} \cup \{u \in V : \deg_{T(e_u)} \geq 2\}. \quad (4)$$

Duplicate source labels are removed. The implementation reconstructs all layer identifiers from the source input, so the forward and recovery processes share no hidden state.

4. Correctness

We prove the main theorem in three steps. The first establishes target legality, the second maps a source witness forward, and the third proves recovery for every qualifying tree.

4.1. Target legality

The graph H is simple by construction. Every set vertex is adjacent to r , and every element vertex e_u is adjacent to s_u because $u \in N_{G[u]}$. The pendant p is adjacent to r , so H is connected. Since $0 \leq k' \leq n$, Equation 3 gives a nonnegative threshold. The forward map always produces a legal target instance.

4.2. From a dominating set to a leaf-rich tree

Suppose the source has a dominating set of size at most k . It also has one, say D , of size at most k' : this is immediate when $k < n$, and V is available when $k \geq n$. Include the edge rp and every edge rs_v . For each $u \in V$, choose one $v \in D$ with $u \in N_{G[v]}$ and include $s_v e_u$.

The resulting graph is connected and has $2n + 1$ edges on $2n + 2$ vertices, so it is a spanning tree. Its leaves include p , all n element vertices, and every set vertex outside D . Their number is at least

$$1 + n + (n - |D|) \geq 2n + 1 - k' = K. \quad (5)$$

So a source witness produces a valid target witness. In particular, a valid target NO-SOLUTION implies source NO-SOLUTION, because Equation 5 would otherwise give a target tree.

4.3. Recovery from an arbitrary target tree

Let T be any target spanning tree with at least K leaves. It has at most

$$(2n + 2) - K = k' + 1 \quad (6)$$

non-leaf vertices. The edge rp belongs to every spanning tree because p has no other neighbor. At least one edge from r to the set layer is also needed to connect the remaining vertices. So r is a non-leaf, leaving at most k' non-leaf vertices outside the root.

Fix $u \in V$. If e_u is a non-leaf, Equation 4 includes u , which dominates itself. If e_u is a leaf, its unique tree neighbor is some s_v . A tree on $2n + 2 \geq 4$ vertices cannot contain adjacent leaves. So s_v is a non-leaf, and Equation 4 includes v . The incidence edge certifies $u \in N_{G[v]}$. In both cases, D_T dominates u .

This argument covers every source vertex, so D_T is dominating. Each member of D_T is charged to a non-root non-leaf of T ; removing duplicate labels can only reduce its size. Equation 6 therefore gives $|D_T| \leq k' \leq k$. Together with the target NO-SOLUTION case, this proves Equation 1 and the main theorem.

5. Complexity and encoding size

If the source has n vertices and m edges, the target has $2n + 2$ vertices. Equation 2 contains one pendant edge, n root edges, and

$$\sum_{v \in V} |N_{G[v]}| = n + 2m \quad (7)$$

incidence edges. The total is $1 + 2n + 2m$. The implementation scans all ordered vertex pairs and uses a source-edge set, so F runs in $O(n^2 + m)$ arithmetic operations. Comparing k with n is polynomial in their bit lengths. The target encoding has length $O((n + m) \log n)$; the possibly long encoding of k is not copied.

Recovery scans the $2n + 1$ tree edges and the two layers. It runs in $O(n)$ arithmetic operations on $O(\log n)$ -bit identifiers and emits at most n identifiers. Both modes disable CPython's process-local decimal integer digit cap before parsing. They therefore accept legal integers of arbitrary finite encoded length while retaining exact arithmetic.

6. Related work and scope

The equivalence between internal vertices of a spanning tree and connected domination is classical; Caro, West, and Yuster state the identity between the maximum leaf number and the complement of the connected domination number [2]. Li and Toulouse prove hardness for a bipartite leaf variant through a Set Cover incidence construction [3, Theorem 2.1]. Specializing sets to closed neighborhoods gives the core two-layer idea used here.

The present construction adds the forced root accounting and an explicit decoder for either layer. The result should be read as a reviewed reconstruction of a search reduction, not as a new NP-hardness theorem. It does not improve target size below $2n + 2$ vertices, optimize the $O(n^2)$ implementation, or formalize the proof in a proof assistant. None of those extensions is needed for the fixed search contract.

7. Conclusion

The incidence construction resolves the ordinary-versus-connected domination gap without changing the source semantics. A pendant forces one internal root, and all remaining internal target vertices map to a dominating set within the source budget. The same counting argument handles every qualifying tree and the exact NO-SOLUTION output. The reduction is explicit, deterministic, polynomial, and executable on unbounded encoded thresholds.

References

- [1] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. Problem ND2, p. 206.
- [2] Y. Caro, D. B. West, and R. Yuster. “Connected Domination and Spanning Trees with Many Leaves.” *SIAM Journal on Discrete Mathematics* 13(2), 202–211, 2000. doi:10.1137/S0895480199353780.
- [3] P. C. Li and M. Toulouse. “Variations of the Maximum Leaf Spanning Tree Problem for Bipartite Graphs.” *Information Processing Letters* 97(4), 129–132, 2006. doi:10.1016/j.ipl.2005.10.011.
- [4] S. Fenner, F. Green, S. Homer, A. L. Selman, T. Thierauf, and H. Vollmer. “Complements of Multi-valued Functions.” *Chicago Journal of Theoretical Computer Science* 1999(3), 1999. Definition 2. Primary text.

A Verification and reproducibility

The proof establishes the general result. The checks below provide finite regression and counterexample evidence only. They use no wall-clock, solver, or subprocess timeout.

A.1 Environment and artifacts

The retained environment used uv 0.12.7 with locked CPython 3.14.2. Typst 0.15.1 compiled this paper. The source and target oracles use only the Python standard library. The executable maps are in `work/algorithm.py`; independent preparation and verification live in `work/check.py` and `work/verify.py`. The dependency metadata are `pyproject.toml` and `uv.lock` at the repository root.

A.2 Commands

From the repository root, run:

```
uv sync --locked
uv run python campaigns/dominating-set-leaf-spanning-tree/work/check.py --self-test
uv run python campaigns/dominating-set-leaf-spanning-tree/work/check.py \
  --candidate campaigns/dominating-set-leaf-spanning-tree/work/algorithm.py
uv run python campaigns/dominating-set-leaf-spanning-tree/work/verify.py \
  --candidate campaigns/dominating-set-leaf-spanning-tree/work/algorithm.py
typst compile campaigns/dominating-set-leaf-spanning-tree/work/paper/
manuscript.typ \
  campaigns/dominating-set-leaf-spanning-tree/work/paper/manuscript.pdf
```

The default candidate mode reads one source JSON instance from standard input and emits the target instance. The extraction mode is

```
python3 campaigns/dominating-set-leaf-spanning-tree/work/algorithm.py --extract
```

It reads an object with keys `source` and `target_solution` and emits one source output.

A.3 Retained results and limits

The prepared loop passed 11 source instances and all 97 valid target outputs enumerated for their constructed instances. The additional checker covered all 44 connected labeled simple source graphs through four vertices and 255 source threshold instances. It exercised 598 recovered target outputs after examining 1,295,656 target edge subsets and 166,307 spanning trees. A separate raw-input regression passed 5,000-digit k values in both candidate modes. The focused independent review extended this boundary check to 4,301, 5,000, and 10,000 digits.

The finite graph domain ends at four source vertices, and positive instances use at most three independently obtained target trees each in the broader checker. The first independent review found the integer-parser boundary defect. After the repair, a focused review returned **advance** and reused its prior correctness, novelty, and significance audits. The reviewer ran in a fresh child context; filesystem confinement and no-delegation were instruction-only, and the backend model identifier was not exposed.